Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM302 Үйлдлийн систем

Linux цөмийн модулиар процессын жагсаалт ба холбоосыг дүрслэх

Linux Kernel Module for Listing Tasks

Бие даалт №1

Хэрэглэгчийн гарын авлага

Шалгасан багш: Г.Ганбат

Багийн дугаар: IV-2-3

Гүйцэтгэсэн оюутнууд: А.Тэмүүлэн /B242270063/

Ч.Балгандамба /B232270142/

Д.Азмаа /B241940101/

Н.Билэгт-Очир /B231910009/

Т.Түвшинтөр /B190910820/

Б.Өсөхбаяр /B210930901/

Улаанбаатар хот

2026 он

1. Зорилго ба дэмжих орчин

TaskGraph нь Linux-ийн process, thread-ийн холбоос, төлөвийг граф болон хүснэгтээр харуулна. Цөмийн модуль нь ажиллаж байгаа task-уудын мэдээллийг цуглуулж, интерфэйс нь тэдгээрийн эцэг-хүүхэд холбоо болон төлөвийг үзүүлнэ.

Горим	Шаардлагатай орчин
Linux UI	Python 3, `python3-tk`; `run.sh`. Туршсан: Python 3.12, Tk 8.6.
Бодит module	Ubuntu 24.04 LTS, GCC/Make, ажиллаж буй kernel-ийн headers. Туршсан: Linux 6.8.0-31, GCC 13.3, procps-ng 4.0.4.

2. Суулгах ба ажиллуулах

2.1. Ubuntu дээр module ажиллуулах

Ubuntu Terminal-д dependency суулгаад төслийн хавтас руу орно:

```
sudo apt update
sudo apt install -y build-essential \
  linux-headers-$(uname -r) python3 python3-tk git procps
REPO=https://github.com/Temka0511/
REPO="${REPO}FCSA_302-IV-2-3-LINUX-"
REPO="${REPO}Kernel-Module-For-Task-Information.git"
git clone "$REPO" taskgraph
cd taskgraph
bash scripts/check_env.sh
make -C src/kernel
sudo insmod src/kernel/taskgraph.ko \
  max_tasks=4096 log_on_load=1
bash run.sh --kernel
```

Командуудыг дээрх дарааллаар ажиллуулна. Төсөл өмнө нь татагдсан бол `cd ~/taskgraph`, дараа нь `git pull --ff-only` ажиллуулж шинэчилнэ. Дараагийн бүх командыг төслийн хавтас дотроос ажиллуулна. `make` нь модулийг хөрвүүлж, `insmod` нь цөмд ачаална. `bash run.sh --kernel` нь интерфэйсийг нээнэ; эх үүсвэр **KERNEL** байна. UI-г энгийн хэрэглэгчээр ажиллуулна. Модуль ачаалах, буулгахад sudo хэрэгтэй. Дуусгаад `sudo rmmod taskgraph` командаар буулгана. **Параметр:** `max_tasks=16..16384`, анхны утга 4096; `log_on_load=1` нь LINEAR/DFS log хэвлэнэ. Өөр kernel ашиглавал өөрийн headers-тай дахин build хийнэ.

3. UI-ийн удирдлага

Удирдлага	Үйлдэл
Эх үүсвэр	`kernel`: module; `procfs`: Linux оношилгоо; `file`: хадгалсан snapshot.
DFS / LINEAR	Task үзүүлэх дарааллыг солино.
Root PID	Тухайн task ба дэд мод; хоосон бол бүгд.
Threads	Нэмэлт thread-үүдийг харуулна эсвэл нууна.
Зангилааны хязгаар	1–2000 хүртэл task харуулна.
Шинэчлэх / Авто шинэчлэлт	Эх үүсвэрийг дахин уншина; 500–10000 ms интервалтай.
Өмнөх / Дараах / Тоглуулах	Нэг snapshot-ийн traversal-ийг алхмаар үзүүлнэ.
Reset	Шүүлт, дараалал, playback-ийг эхний байдалд оруулна.
Файл нээх / Хадгалах	Бүтэн TSV snapshot импортлох, экспортлох.

Зангилаа дээр дарвал PID, PPID, TGID, төлөв, дарааллын дэлгэрэнгүй гарна. Scroll болон дунд товчоор графыг шилжүүлнэ. Ногоон R, цэнхэр S, улбар шар T/D, саарал I, ягаан Z/X төлөвийг илэрхийлнэ. Улбар хүрээ нь сонгосон task. Тасархай сум нь thread-ийн холбоос. LINEAR сонголтод хүснэгт, алхам болон зангилааны дугаар LINEAR дарааллаар явна; графын салбарын байршил parent-child холбоог уншихад зориулж DFS дарааллаар хэвээр байрлана.

4. Ашиглах жишээ

4.1. Жишээ 1: бэлэн модыг судлах

1. Ubuntu Terminal-д дараах командыг ажиллуулж, Reset дарна.

```
bash run.sh --source examples/small_tree.tsv
```

2. DFS сонгож Тоглуулах дарна. Дараалал: 0, 1, 10, 11, 20, 21, 2.
3. Root PID=10 оруулаад Хэрэглэх дарна. 10, 11, 20, 21 гэсэн 4 task харагдана.
4. Threads-ийн чагтыг авахад 11 worker нуугдана.

4.2. Жишээ 2: kernel-ийн хадгалсан хэмжилт

Файл нээхээр `docs/evidence/created.tsv` сонгож, Root PID=90 оруулна. 7 task бүхий мод гарна. Дараа нь `stopped.tsv` нээхэд root-ийн төлөв T болж улбар шар болно. `after.tsv` нээхээс өмнө Root PID-г цэвэрлэнэ: workload аль хэдийн дууссан тул PID 90 байхгүй. File горимын авто шинэчлэлт нь ижил файлыг дахин уншина; бодит процессын шинэ өөрчлөлтийг хэмжихгүй.

4.3. Жишээ 3: бодит Linux процессын мод

Module болон UI-г kernel горимд ажиллуулсны дараа өөр Terminal нээж, `cd ~/taskgraph` командаар төслийн хавтас руу орно:

```
export PYTHONPATH="$PWD/src"
python3 -m taskgraph.workload \
  --depth 2 --fanout 2 --seconds 300 &
ROOT_PID=$!
echo "ROOT_PID=$ROOT_PID"
```

ROOT_PID мөрийн дугаарыг UI-д оруулж, Шинэчлэх, Хэрэглэх дарна. Энэ workload нь 1+2+4=7 process үүсгэнэ; 300 секунд ажиллана. Авто шинэчлэлтээр хугацаа дуусахад task-ууд алга болохыг ажиглана. Root өөрөө дуусвал талбарыг цэвэрлэнэ. Depth=0..3, fanout=1..3, seconds=2..300. SIGSTOP/CONT турших бол зөвхөн өөрийн workload-ийн root PID-д дараах дохиог өгнө:

```
kill -STOP "$ROOT_PID"
# Refresh in the UI, then continue:
kill -CONT "$ROOT_PID"
```

Дээрх хувьсагчийг үүсгэсэн тэр Terminal-д STOP/CONT командыг ажиллуулна. `"$ROOT_PID"` нь процессын дугаарыг автоматаар авна; UI-ийн Root PID талбарт хэвлэгдсэн тоог оруулна. STOP өгөөд Шинэчлэх дарахад root-ийн төлөв T болж улбар шар болно. CONT өгсний дараа дахин шинэчлэхэд task үргэлжлэн ажиллана. Бусад системийн task-д дохио өгөхгүй.

5. Үйлдлүүдийг шалгах

1. **LINEAR ба DFS.** Root PID-г цэвэрлээд DFS сонгож Дараах дарна. Эцэг task, түүний дэд мод, дараагийн салаа гэсэн дараалалтай байна. LINEAR нь kernel-ийн шугаман дарааллыг үзүүлнэ.
2. **Threads.** Created snapshot-д Root PID=90 үед 7 task байна. Threads-ийг унтраахад PID 95, 96 нуугдаж 5 process үлдэнэ. Python workload нэмэлт thread үүсгэхгүй.
3. **Хадгалах ба нээх.** TSV snapshot хадгалаад дахин нээхэд PID, холбоос, төлөв ижил байна. Бодит хэмжилтэд буцахдаа цонхоо хааж `bash run.sh --kernel` ажиллуулна.
4. **Буруу оролт.** Root PID-д abc, зангилааны хязгаарт 0 оруулахад алдаа мэдээлнэ. Утгыг заасан мужид засаж Хэрэглэх дарна.
5. **Дуусах ба Reset.** Workload дуусвал Root PID-г цэвэрлэж шинэчилнэ. Reset харагдацыг сэргээнэ; системийн процессыг зогсоохгүй.

6. Ажиллагааны шалгалт

6.1. Үндсэн build ба тест

```
make -C src/kernel
bash scripts/test.sh
sudo dmesg | grep 'taskgraph LINEAR' | head
sudo dmesg | grep 'taskgraph DFS' | head
ps -el
ps -eLf
cat /proc/taskgraph
```

Python тестийн хүлээгдсэн үр дүн нь Linux дээр 30 тест OK. LINEAR log-ийг process leader-уудтай, DFS log-ийг thread-үүдтэй харьцуулна. PID 0 нь `init_task`. Хэмжилтийн агшин өөр учраас түр task, төлөвийн зөрүү гарч болно.

6.2. Нэмэлт live ба QEMU тест

```
PYTHONPATH=src python3 scripts/verify_live.py
sudo apt install -y qemu-system-x86 busybox-static procps
PYTHONPATH=src python3 scripts/qemu_smoke.py \
  --kernel /boot/vmlinuz-$(uname -r)
```

Эхний скрипт ачаалсан module дээр 8 live шалгалт хийнэ. Хоёр дахь нь тохирох kernel image, build хийсэн module-оор QEMU дотор 17 шалгалт ажиллуулна. Image нь уншигдах боломжтой байна. Алдаа гарвал exit code 1 буцаана. Live тест нь `evidence/ps-el.txt`, `ps-eLf.txt`, `ps_comparison.json` үүсгэнэ. QEMU тестийн ижил нотолгоо `docs/evidence/`-д хадгалагдана. Туршилтын task-уудын PID, PPID, TGID, нэр, төлөв бүгд таарвал comparison-ийн `passed=true` байна. PID 0 болон хэмжилтийн үеийн cat/ps/insmod процессын зөрүүг JSON-д тусад нь харуулна.

7. Түгээмэл алдаа ба хязгаарлалт

Алдаа	Шийдэл
`/proc/taskgraph` алга	Module build, `sudo insmod` хийсэн эсэхийг шалгана.
Invalid module format	`uname -r`-тэй таарсан headers суулгаж clean, build хийнэ.
Key was rejected	Secure Boot идэвхтэй үед module-д MOK гарын үсэг зурна; доорх зааврыг дагана.
File exists	Модуль ачаалагдсан байна. Шинэ код суулгаж байгаа бол UI-г хааж, `sudo rmmod taskgraph` хийгээд дахин ачаална.
`tkinter` олдохгүй	Ubuntu-д `sudo apt install python3-tk`.
DISPLAY алдаа	Desktop session-оос UI нээх эсвэл CLI ашиглана.
PID олдохгүй	Дууссан эсвэл тухайн snapshot-д байхгүй PID. Root талбарыг цэвэрлэнэ.
`truncated=1`	Capture сар хүрсэн. Module-ийг илүү их `max_tasks`-аар дахин ачаална.
Графын task дутуу	View limit болон Threads шүүлтийг шалгана.

7.1. Secure Boot идэвхтэй үед module-д гарын үсэг зурах

`Key was rejected by service` алдаа гарвал `mokutil --sb-state` гэж шалгана. Өөрийн эрхтэй Ubuntu/Debian VM-д дараах алхмыг хийж болно. Түлхүүрээ repository-д оруулахгүй, бусдад дамжуулахгүй.

```
sudo apt install -y mokutil openssl
mkdir -m 700 -p "$HOME/taskgraph-mok"
openssl req -new -x509 -newkey rsa:2048 -nodes \
  -days 3650 -subj "/CN=TaskGraph module/" \
  -addext "extendedKeyUsage=codeSigning,1.3.6.1.4.1.2312.16.1.2" \
  -keyout "$HOME/taskgraph-mok/MOK.priv" \
  -outform DER -out "$HOME/taskgraph-mok/MOK.der"
chmod 600 "$HOME/taskgraph-mok/MOK.priv"
sudo mokutil --import "$HOME/taskgraph-mok/MOK.der"
```

Сүүлчийн командын дараа нууц үг тохируулж, VM-ээ restart хийнэ. Ачаалах үеийн цэнхэр *MokManager* дэлгэцээс *Enroll MOK*-г сонгон дээрх нууц үгээр баталгаажуулж дахин ачаална. Тэгээд төслийн хавтас дотор:

```
sudo /lib/modules/$(uname -r)/build/scripts/sign-file \
  sha256 "$HOME/taskgraph-mok/MOK.priv" \
  "$HOME/taskgraph-mok/MOK.der" src/kernel/taskgraph.ko
sudo insmod src/kernel/taskgraph.ko \
  max_tasks=4096 log_on_load=1
```

Module-ийг дахин build хийвэл дахин sign хийнэ. Энэ нь өөрийн үүсгэсэн түлхүүрийг системд итгэмжлүүлэх үйлдэл тул зөвхөн өөрийн VM дээр хийнэ.

Snapshot авахад `stop_machine()` CPU-уудыг түр саатуулж children/sibling жагсаалтыг хамгаална. `/proc/taskgraph`-ийг бүх хэрэглэгч унших эрхтэй (0444) бөгөөд унших бүрд capture хийдэг: зөвхөн итгэмжлэгдсэн сургалтын VM-д ажиллуулж, давтамжтай уншихаас зайлсхий. Latency мэдрэмтгий production орчинд зориулаагүй. Тусдаа snapshot болон ps хэмжилтүүдийн хооронд төлөв өөрчлөгдөнө. Task-ийн нэр нь богино `comm` утга. PID нь init namespace-ийнх тул module-ийг тусдаа Linux VM-д ашиглана. Хадгалсан файл нь бодит хугацааны өөрчлөлтийг хэмжихгүй.

8. Төслийн файлууд

Зам	Агуулга
`src/kernel/`	C kernel module, Makefile.
`src/taskgraph/`	UI, parser, CLI, workload, launcher.
`tests/`, `examples/`	Автомат тест ба бэлэн өгөгдөл.
`scripts/`	Орчин, live болон QEMU шалгах скрипт.
`docs/evidence/`	Бодит kernel туршилтын log, JSON, TSV.